A BUILD-AND-DEBUG COURSE FOR UDEMY

SystemC & TLM-2.0 in Practice

Build, debug, and tune a tiny virtual SoC, one transaction at a time.

Follow a single fictional edge-sensing device from an empty simulation to a tested transaction-level model. Every module adds a working subsystem component and teaches you to investigate the failures that come with it.

6–7 hours | Short focused videos | One evolving project | Debug missions throughout

The Distinctive Hook

Instead of treating SystemC and TLM-2.0 as a list of APIs, the course follows one build: **EdgeSense-1**, a fictional sensor controller with a firmware-like initiator, address router, SRAM, timer, sensor registers, and an interrupt output.

Students see the same system grow from a first clocked process into a reusable virtual prototype. The story creates a reason to learn each topic: a transaction goes to the wrong device, timing is counted twice, or a fast memory path becomes stale after an update. Students learn to diagnose the model, not just reproduce code.

> **Recurring lesson rhythm:** Observe a symptom → form a hypothesis → model the behavior → trace and test it → make a fidelity or performance decision.

Most lessons are 4–9 minutes. Each ends with a visible result, a quick investigation, or a small design choice. Longer build sessions are split into chapters so students can code alongside the instructor.

Who It's For

- C++ developers moving into hardware modeling or virtual platforms
- RTL, verification, and SoC engineers who want to model above the signal level
- Students and engineers preparing to work with SystemC-based tools and models

Prerequisites

- Basic C++: classes, pointers/references, templates, and compiling a small program
- Digital-systems familiarity is helpful, but not required
- No prior SystemC or TLM experience required

What Students Will Be Able to Do

- Explain SystemC processes, events, scheduling, and simulation time
- Build modules, ports, interfaces, and self-checking testbenches
- Connect TLM-2.0 initiators and targets and construct generic payload transactions
- Implement blocking and non-blocking transport and explain their trade-offs
- Model annotated timing, temporal decoupling, and synchronization
- Create an address map, route transactions, and model register side effects and errors
- Use debug transport and DMI while understanding when each is appropriate
- Trace transaction behavior and debug common modeling mistakes
- Choose a suitable level of model detail for a question the model must answer

Course Structure

Estimated runtime: 6–7 hours. The course uses short concept videos, narrated code walkthroughs, downloadable checkpoints, and occasional longer guided builds.

Three recurring lenses: transaction correctness, timing meaning, and model performance. Students revisit all three as EdgeSense-1 grows.

Downloads: starter and completed project, a transaction trace worksheet, address-map template, bug cards, timing checklist, and capstone acceptance tests.

Curriculum: The EdgeSense-1 Build

0. Cold Open: The Transaction That Vanished

Meet EdgeSense-1 and inspect a failing test	5 min
What does a useful virtual prototype need to answer?	5 min
Three model-detail choices: signal, transaction, loosely timed	7 min

Takeaway: A project map and a question-led way to judge model fidelity.

1. Make Simulation Time Visible

Smallest runnable SystemC program	6 min
Modules, elaboration, and sc_main	7 min
SC_METHOD and SC_THREAD through a timing trace	8 min
Events, sensitivity, and scheduling intuition	8 min
Time units, resolution, and wait()	7 min
Debug mission: why did this process run twice?	7 min

Build output: A runnable simulation and a first annotated timeline. **Challenge:** Predict the output before running the trace.

2. Draw the Machine Before Coding It

Ports, signals, and interfaces in one small component	7 min
Wire up a top-level testbench	7 min
Constructor setup and process registration	6 min
Turn a register block into a reusable module	7 min
Build mission: sensor stub plus testbench	9 min

Build output: EdgeSense-1's sensor block skeleton and a clean testbench. **Design choice:** Which behavior belongs in a signal, and which belongs in a transaction?

3. Send the First Transaction

Why model a transaction instead of every bus wire?	6 min
Generic payload: command, address, data, response	8 min
Initiator and target sockets, drawn before bound	8 min
Implement b_transport in a tiny SRAM	9 min
Read, write, response status, and invalid commands	8 min
Debug mission: the address is right, but the data is not	8 min

Build output: A tested initiator-to-SRAM read/write path. **Challenge:** Add a failing test first, then fix the target.

4. Put Meaning Into Time

Annotated delay: what the number means	7 min
Return now or wait inside the target?	8 min
Account for latency exactly once	7 min
Timing experiment: compare two memory models	8 min
Debug mission: find the double-counted delay	8 min

Build output: A timing contract for SRAM and a test that checks it. **Design choice:** Decide what timing detail the use case actually needs.

5. Give Every Device an Address

Sketch EdgeSense-1's address map	6 min
Decode and route a transaction to multiple targets	9 min
Model timer and sensor registers with side effects	9 min
Bad addresses, response errors, and boundary tests	8 min
Debug mission: an address alias hides in the map	8 min
Guided build: connect the subsystem	12 min

Build output: A routed subsystem with SRAM, timer, and sensor registers. **Challenge:** Design a boundary test that catches accidental overlap.

6. Follow a Transaction Across Time

When blocking transport stops answering the question	6 min
Non-blocking transport and its two paths	8 min
Phases and synchronization, visualized as a trace	9 min
Build a minimal non-blocking target	10 min
Debug mission: a phase arrives in the wrong state	8 min

Build output: A small non-blocking transaction with a phase-by-phase trace. **Design choice:** Explain why this path is useful for the model, not just how to implement it.

7. Make It Faster Without Making It Wrong

Temporal decoupling and local time	8 min
Quantum keeper: synchronization in context	8 min
DMI: when skipping transactions is worthwhile	7 min
DMI permissions, invalidation, and stale data	9 min
Debug transport for inspection and testing	6 min
Performance mission: measure before and after	8 min

Build output: A measured fast-memory experiment and notes on correctness boundaries. **Challenge:** Explain when the optimization should be turned off.

8. Capstone: EdgeSense-1 Under Test

Read the acceptance tests and plan the final model	6 min
Complete the initiator and transaction helpers	9 min
Integrate SRAM, timer, sensor, and router	12 min
Add documented timing and useful trace output	9 min
Run the mystery-transaction challenge	8 min
Review: what does this model promise to represent?	7 min

Deliverable: A reusable, tested subsystem and a short model contract describing its address map, timing assumptions, and known limits.

9. Where to Go Next

Model review checklist: correctness, timing, performance	6 min
How to extend the project without losing clarity	5 min
Pick the next model-detail level for your own problem	5 min

Capstone: Definition of Done

- Builds and runs using the course's documented toolchain
- Separates initiator, interconnect, and target responsibilities
- Passes valid read/write and invalid-address tests
- Documents its address map and timing assumptions
- Includes trace output that helps explain a transaction
- Explains at least one performance/fidelity trade-off and its correctness risk

Udemy Course Description

> **Don't just learn the TLM APIs. Build a tiny virtual SoC, then learn to debug what it tells you.**

SystemC and TLM-2.0 are powerful tools for modeling systems above the signal level. But learning sockets, payloads, simulation time, and transaction phases can feel abstract when every example is disconnected.

In this hands-on course, you'll follow one project from its first simulation to a tested virtual prototype: EdgeSense-1, a small sensor controller with SRAM, a timer, memory-mapped registers, and an interconnect.

Each short lesson gives you a concrete reason to learn the next concept. You'll inspect traces, predict behavior, fix deliberately introduced modeling bugs, and make choices about timing and performance. You'll implement blocking and non-blocking transport, explore temporal decoupling and DMI, and test the system with a growing set of checks.

You'll finish with more than a collection of examples: you'll have a working subsystem, a model contract, and a repeatable method for reasoning about transaction correctness, timing, and fidelity.

Basic C++ is recommended. Prior SystemC experience is not required.

Promotional Video Script

SystemC & TLM-2.0 in Practice | Course design

A transaction reaches the wrong device. A read comes back with the wrong value. And the timing trace doesn't match what you expected. Those are exactly the kinds of problems we'll learn to solve.

In this course, we'll build EdgeSense-1, a small virtual sensor system, one piece at a time. You'll learn SystemC simulation basics, connect TLM initiators and targets, add an address router, model timing, and explore non-blocking transport and performance features.

Every lesson ends with something you can run, inspect, or debug. By the end, you'll have a tested subsystem and a practical way to reason about the models you build.

If you know basic C++ and want to make SystemC and TLM-2.0 feel concrete, let's get started.

Student Engagement Features

- **Bug cards:** Short broken examples on scheduling, address decode, delay handling, phases, and DMI invalidation
- **Predict-before-run pauses:** Students commit to an expected trace before seeing the result
- **Model contract:** A one-page record of timing promises, address map, fidelity, and limitations
- **Checkpoint tests:** Small tests students can run after each project milestone
- **Two completion tracks:** Follow-along build for beginners; optional extension challenges for experienced engineers

Course Assets and Production Notes

- Starter project, completed project, and tagged checkpoint versions
- EdgeSense-1 architecture and address-map diagrams
- Printable transaction trace worksheet and simulation timeline reference
- Bug cards with separate hints and solutions
- Capstone tests and a model-contract template
- Short glossary: initiator, target, socket, payload, phase, DMI, and temporal decoupling

> **Recording note:** Pin and test the exact SystemC, compiler, and build-tool versions used in the lessons. Show the same project running from a clean checkout and document the supported setup platforms.